SQL Practice Assignment

WHERE | GROUP BY & HAVING | Combined Queries | JOINS

Name: _____ **Date:** _____

Batch / ID: _____ **Section:** _____

Instructions

- Write all queries in standard SQL (ANSI-style). Use table aliases wherever more than one table is involved.
- Use only the tables described below. Column names must match exactly.
- Write the query in the answer box and, where asked, a one-line explanation of your approach.

Database Schema

employees
emp_id (PK), first_name, last_name, job_title, salary, hire_date, city, dept_id (FK), manager_id (FK to employees.emp_id, NULL if no manager)

departments
dept_id (PK), dept_name, location

customers
customer_id (PK), customer_name, city, country

orders
order_id (PK), customer_id (FK), emp_id (FK, salesperson), order_date, amount, status (status values: 'Placed', 'Shipped', 'Delivered', 'Cancelled')

Section A - WHERE Clause (5 Questions)

Q1.

List the **first_name**, **last_name** and **city** of all employees who work in **'Chennai'**.

Q2.

Retrieve all orders where the **amount** is greater than **5000**. Show order_id, order_date and amount.

Q3.

Display employees who were hired between **2022-01-01** and **2023-12-31** and earn a **salary of at least 60000**. Sort the result by salary in descending order.

Q4.

Find employees whose **job_title** is one of **'Data Engineer', 'BI Developer' or 'Analyst'** and whose **last_name starts with 'S'**. (Use IN and LIKE.)

Q5.

Retrieve employees who are either (a) **top-level** (have no manager) or (b) earn **more than 90000** - but in both cases **exclude department 40** and any employee whose job_title **contains the word 'Intern'**. Handle NULLs correctly and use parentheses to control AND/OR precedence.

Section B - GROUP BY & HAVING (5 Questions)

Q1.

Count the number of employees in each department. Show dept_id and employee_count.

Q2.

Calculate the **total order amount** for each customer_id. Show customer_id and total_amount.

Q3.

Show the departments (dept_id) whose **average salary is greater than 70000**. Display dept_id and avg_salary rounded to 2 decimals.

Q4.

Find customers who have placed **more than 3 orders** and whose **total order amount exceeds 20000**. Display customer_id, order_count and total_amount.

Q5.

For each combination of **order year** and **status**, display the number of orders and the average amount. Keep only groups with **at least 5 orders** whose average amount is **higher than the overall average order amount of the entire table** (use a subquery inside HAVING). Sort by year.

Section C - WHERE + GROUP BY + HAVING Together (3 Questions)

Q1.

Among employees hired **after 2021-01-01**, count the employees in each **city**. Show only cities having **at least 2** such employees.

Q2.

Considering only orders with status **'Delivered'**, find the total amount per customer. Keep customers whose total is **above 15000**, sort by total descending and return the **top 5** customers.

Q3.

Exclude employees whose job_title is **'Intern'** or whose salary is NULL. For each **dept_id and job_title**, display the headcount, minimum salary, maximum salary and salary spread (max - min). Keep only groups with **at least 2 employees** and a spread of **more than 20000**. Order by spread descending.

Section D - JOINS (5 Questions)

Q1.

Using an **INNER JOIN**, display each employee's full name (first_name and last_name combined) along with the **dept_name** of their department.

Q2.

Using a **LEFT JOIN**, list **all customers** with their orders. Customers who have never placed an order must still appear. Show customer_name, order_id and amount.

Q3.

Using a **RIGHT JOIN**, list every department with the number of employees in it. Departments with **no employees** must appear with a count of 0.

Q4.

Using a **SELF JOIN**, display each employee's name along with their **manager's name**. Employees without a manager should show **'No Manager'**.

Q5.

Using **INNER and LEFT JOINs** across multiple tables, show, for every department, the dept_name and the **total amount of 'Delivered' orders** handled by its employees (orders.emp_id). Departments with no delivered orders must still appear with a total of 0. Sort by total descending. *Hint: think about where the status filter must go so the LEFT JOIN is not turned into an INNER JOIN. Explain your choice in one line.*